



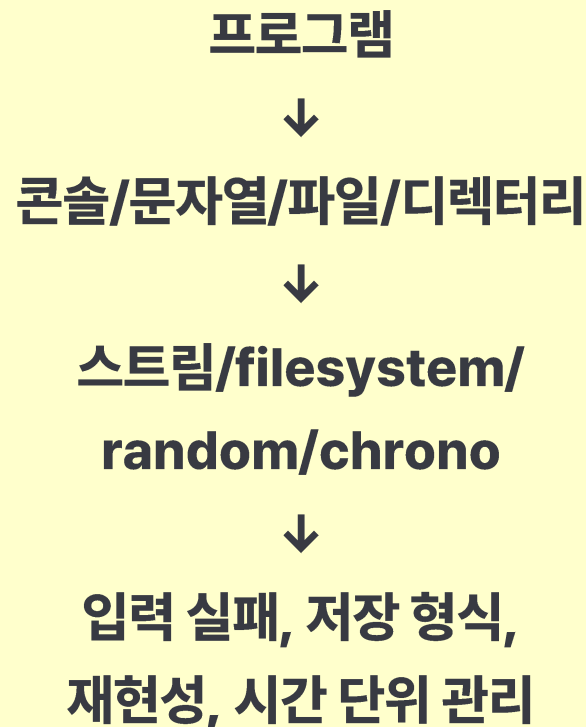
15. 입출력과 표준 유틸리티

Heesung Oh

<https://github.com/3dapi>



- 스트림 입출력
- 문자열 스트림
- 형식화된 출력
- 텍스트 파일 입출력
- 바이너리 파일 입출력
- 파일 시스템
- 난수
- 시간





- operator>>는 타입에 맞게 문자를 해석
- int에는 정수 형식 필요
- std::string은 앞쪽 공백을 건너뛰 뒤 공백 전까지 읽음
- 공백을 포함한 한 줄은 std::getline() 사용

```
int level = 0;  
std::string name;  
  
std::cin >> name >> level;
```

```
// 입력: Dark Knight 20  
std::string name;  
int level = 0;  
  
std::cin >> name >> level;  
// name → "Dark"  
// level → "Knight"에서 입력 실패  
  
std::string fullName;  
std::getline(std::cin, fullName);
```





- 스트림은 입출력 결과를 상태 플래그로 기록
- 실패한 입력 스트림은 복구 전까지 후속 입력을 수행 안함

```
if (int value = 0; std::cin >> value)
{
    std::cout << "Input: " << value << '\n';
}
else
{
    std::cout << "Invalid input\n";
}
```

- fail()은 failbit 또는 badbit 확인
- 상태 검사 함수는 완전히 배타적이지 않음

상태	의미
std::ios::goodbit	오류 상태 없음
std::ios::eofbit	입력 과정에서 파일 끝 도달
std::ios::failbit	형식 변환 실패 등 복구 가능한 오류
std::ios::badbit	장치 오류 등 심각한 오류



- 값을 읽는 연산 자체를 반복 조건으로 사용
- 마지막 값 처리 뒤의 읽기 실패에서 EOF가 설정될 수 있음
- EOF가 아닌 종료는 형식 오류나 장치 오류일 수 있음

```
int value = 0;
```

```
while (input >> value)  
{  
    Process(value);  
}
```

```
// 권장하지 않음
```

```
while (!input.eof())  
{  
    input >> value;  
    Process(value);  
}
```

```
// 반복 종료 원인 확인
```

```
while (input >> value)  
{  
    Process(value);  
}  
if (!input.eof())  
{  
    std::cerr << "Invalid data format\n";  
}
```



- 잘못된 입력을 다시 받으려면
상태 플래그와 입력 버퍼를 모두 정리
→ 잘못된 문자가 버퍼에 남으면 입력이 계속 실패
- `clear()` → 오류 플래그 초기화
- `ignore()` → 버퍼의 남은 잘못된 입력 제거

```
#include <iostream>
#include <limits>

int ReadInteger()
{
    int value = 0;
    while (!(std::cin >> value))
    {
        std::cout << "정수를 입력하세요: ";

        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
    }
    return value;
}
```





- 출력 스트림은 형식 상태를 보관
- 진법, 정렬, 폭, 채움 문자, 정밀도 설정 가능
- 일부 조정자는 변경할 때까지 유지
- `std::setw()`는 바로 뒤의 한 필드에만 적용
- 라이브러리 성격의 출력 함수는 스트림 상태 복원 검토

```
#include <iomanip>
#include <iostream>

int value = 255;
std::cout << std::dec << value << '\n';
std::cout << std::hex << value << '\n';
std::cout << std::oct << value << '\n';

std::cout << std::hex << 255 << '\n';
std::cout << 16 << '\n';    // 10
std::cout << std::dec << 16 << '\n';
```





- 입력 실패 시 객체가 일부만 변경되지 않도록 임시 변수에 읽음
- 모든 입력과 의미 검사가 성공한 뒤 객체 갱신
- failbit는 형식 자체의 의미 오류를 표현할 때 사용 가능
- 저장 형식은 구분자, 공백, 버전 규칙 필요

```
std::istream& operator>>(std::istream& input, Item& item)
{
    int id = 0;
    std::string name;
    int count = 0;
    if (!(input >> id >> name >> count)) {
        return input;
    }
    if (id <= 0 || count < 0) {
        input.setstate(std::ios::failbit);
        return input;
    }

    item = Item{id, std::move(name), count};
    return input;
}
```



- 출력 스트림은 데이터를 버퍼에 모은 뒤 장치로 보낼 수 있음
- 작은 출력마다 장치 접근을 반복하지 않아 성능에 유리
- 즉시 출력이 필요하면 `std::flush`
- 반복 출력에서는 불필요한 `std::endl` 사용 지양

```
std::cout << "Loading...";  
std::cout << "Loading..." << std::flush;
```

```
std::cout << "Completed" << std::endl;  
std::cout << "Completed\n";
```

```
for (int i = 0; i < 10000; ++i)  
{  
    std::cout << i << '\n';  
}
```

표현	의미
<code>'\n'</code>	줄바꿈 문자 출력
<code>std::flush</code>	버퍼 강제 출력
<code>std::endl</code>	줄바꿈 + flush



- `operator>>`는 값을 읽은 뒤 구분 문자를 버퍼에 남길 수 있음
- 정수 입력 뒤 `getline()`을 바로 호출하면 남은 줄바꿈만 읽을 수 있음
- `std::ws`는 줄바꿈뿐 아니라 이어지는 모든 공백 제거
- 앞쪽 공백 보존이 필요하다면 `ignore()` 방식 검토

```
int age = 0;
std::string name;

std::cin >> age;
std::getline(std::cin, name);

// 현재 행의 나머지 제거
std::cin >> age;

std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
std::getline(std::cin, name);

// 앞쪽 공백 제거
std::cin >> age;
std::getline(std::cin >> std::ws, name);
```



- `std::string`을 입력 또는 출력 장치처럼 사용
- 문자열 파싱
- 메시지 조립
- 테스트 코드에서 활용
- 콘솔이나 파일 없이 스트림 형식 변환 사용 가능

```
#include <sstream>
#include <string>

std::string text = "1001 Potion 10";
std::istringstream input{text};

int id = 0;
std::string name;
int count = 0;

if (input >> id >> name >> count)
{
    // id = 1001, name = "Potion", count = 10
}
```

- 문자열 안의 공백은
형식 입력에서 구분자로 사용



- 행 전체를 읽고 내부를 구분자로 분리
- 숫자 필드는 별도 변환 필요

```
std::string record = "1001,Potion,10";  
std::istream input{record};
```

```
std::string idText;  
std::string name;  
std::string countText;
```

```
std::getline(input, idText, ',');  
std::getline(input, name, ',');  
std::getline(input, countText);
```

- 형식이 단순하고 빠른 숫자 변환이 필요하면 `from_chars()` 검토
- 스트림 방식은 여러 타입을 같은 문법으로 처리하기 쉬움

변환 방식	특징
문자열 스트림	여러 타입, 공백 규칙 처리 편리
<code>std::from_chars()</code>	단순 숫자 변환, 예외 없음



- 입력 성공만 확인하면 일부 문자열만 변환될 수 있음
- "100abc"에서 정수 100을 읽는 시점까지는 성공
- 전체가 형식에 맞는지 확인하려면 남은 문자 검사

```
std::optional<int> ParseInteger(const std::string& text)
{
    std::istream input{text};
    int value = 0;
    if (!(input >> value))
    {
        return std::nullopt;
    }

    input >> std::ws;
    if (!input.eof())
    {
        return std::nullopt;
    }

    return value;
}
```

- `std::ws`는 끝의 공백 제거
- 실제 추출 결과를 기준으로 오류 처리



- 값의 의미를 바꾸지 않고 출력 표현을 정함
- 표, 로그, 사용자 인터페이스 가독성 개선
- 스트림 조정자와 `std::format()` 활용

```
#include <iomanip>
#include <iostream>
```

```
std::cout << std::left
  << std::setw(12) << "Name"
  << std::right
  << std::setw(8) << "Score"
  << '\n';
```

```
std::cout << std::left
  << std::setw(12) << "Knight"
  << std::right
  << std::setw(8) << 1500
  << '\n';
```

```
// 출력
// Name      Score
// Knight    1500
```



- 형식 문자열에 값을 대입해 std::string 생성
- 스트림 상태를 변경하지 않음
- 호출 하나의 형식이 문자열 안에 표현

```
#include <format>
#include <string>

std::string name = "Knight";
int level = 12;

std::string text = std::format(
    "Name: {}, Level: {}",
    name,
    level);

std::string text = std::format(
    "{1} has level {0}",
    level,
    name);
```

- {}는 교체 필드
- 인수 위치를 명시할 수 있음
- 자동 인덱스와 수동 인덱스를 한 형식 문자열에서 섞지 않음



15.3.3 형식 문자열과 서식 지정

서식	의미
폭	최소 출력 너비
채움 문자	남은 칸을 채울 문자
정렬	왼쪽, 오른쪽, 가운데
정밀도	부동소수점 자릿수
진법	2진, 8진, 10진, 16진

```
std::format("{", 42);  
std::format("{:8}", 42);  
std::format("{:08}", 42);  
std::format("{:+}", 42);  
std::format("{:#x}", 255);
```

```
std::format("{:<10}", "left");  
std::format("{:>10}", "right");  
std::format("{:^10}", "center");  
std::format("{:*^10}", "title");
```

```
std::format("{:.2f}", 12.3456);  
std::format("{:.3e}", 12.3456);  
std::format("{:8.2f}", 12.3456);
```



- 텍스트 파일은 데이터를 문자 형태로 저장
- 사람이 읽고 수정하기 쉬움
- 구분자, 인코딩, 형식 규칙 필요
- 숫자를 문자열로 변환하는 비용 존재

```
#include <fstream>
#include <string>
```

```
std::ofstream output{"player.txt"};
if (!output)
{
    return 1;
}
output << 1 << ' ' << "Knight" << ' ' << 100 << '\n';
```

```
std::ifstream input{"player.txt"};
if (!input)
{
    std::cerr << "Failed to open player.txt\n";
}
```

- 파일 스트림 객체 생성 & 파일 오픈
- 열기 실패 시 스트림은 실패 상태



모드	의미
<code>std::ios::in</code>	읽기 허용
<code>std::ios::out</code>	쓰기 허용
<code>std::ios::app</code>	모든 쓰기를 파일 끝에서 수행
<code>std::ios::ate</code>	파일을 연 직후 위치를 끝으로 이동
<code>std::ios::trunc</code>	기존 내용 제거
<code>std::ios::binary</code>	텍스트 변환 없이 바이트 단위 처리

```
std::ofstream output{"log.txt"};
```

- 기본 `ofstream`은 기존 파일 내용을 자를 수 있음

```
std::ofstream output{"log.txt", std::ios::out | std::ios::app};
```

- 기존 내용 뒤에 추가



- 한 행이 하나의 레코드이면 `std::getline()` 사용
- 줄바꿈 문자는 결과 문자열에 포함되지 않음
- 빈 행도 정상 데이터일 수 있음
- 행 번호를 기록하면 오류 위치 추적이 쉬움

```
std::ifstream input{"config.txt"};
std::string line;

while (std::getline(input, line)) {
    if (line.empty()) {
        continue;
    }
    ProcessLine(line);
}

if (!input.eof()) {
    std::cerr << "Failed while reading config.txt\n";
}

if (!line.empty() && line[0] == '#') {
    continue;
}
```



```
std::string line = "1001,Potion,10";  
std::istringstream input{line};
```

```
std::string idText;  
std::string name;  
std::string countText;
```

```
if (std::getline(input, idText, ',') &&  
    std::getline(input, name, ',') &&  
    std::getline(input, countText))  
{  
    // 세 필드 읽음  
}
```

- 텍스트 파일은 먼저 행을 읽고
 행 내부를 분리하면
 오류 위치 추적이 쉬움
- 숫자 필드는 별도 변환
- 필드 수와 변환 성공 여부 확인 필요

items.csv

↓ getline(file, line)

한 행

↓ getline(record, field, ',')

필드 분리

↓

타입 변환과 검증





- 파일 열기 실패
- 읽기 실패
- 쓰기 실패
- 형식 오류
- 장치 오류

```
std::ifstream input{"items.txt"};
if (!input.is_open()) {
    std::cerr << "File open failed\n";
    return;
}

Item item{};
while (input >> item)
{
    items.push_back(item);
}

if (input.bad()) {
    std::cerr << "Device error while reading\n";
}
else if (!input.eof()) {
    std::cerr << "Invalid item format\n";
}
```

- `is_open()`은 파일 연결 여부 확인
- 스트림 상태는 최근 입출력 연산 성공 여부 표현



- 값을 문자로 변환하지 않고 바이트 형식으로 저장
- 파일 크기, 변환 비용 축소
- 저장 형식 크기, 바이트 순서, 버전 호환성 직접 정의

모드	역할
텍스트 모드	운영체제에 따라 줄바꿈 변환 가능
바이너리 모드	기록한 바이트와 읽은 바이트 사이의 텍스트 변환 방지

```
std::ofstream output{"data.bin", std::ios::out |  
std::ios::binary};  
std::ifstream input{ "data.bin", std::ios::in | std::ios::binary};
```

- 바이너리 모드가 객체를 자동 직렬화 안함
- 저장과 복원 규칙은 프로그램이 정의



15.5.2 read와 write

```
#include <stdint>
#include <fstream>

std::int32_t score = 1500;

std::ofstream output {"score.bin", std::ios::binary};

output.write(reinterpret_cast<const char*>(&score), sizeof(score));
```

```
std::int32_t score = 0;

std::ifstream input {"score.bin", std::ios::binary};

input.read(reinterpret_cast<char*>(&score), sizeof(score));
if (!input)
{
    std::cerr << "Failed to read score\n";
}
```





- 고정 크기 정수형은 파일 필드 크기 명시에 도움
- 메모리 그대로 기록하면 바이트 순서는 여전히 환경 의존
- 파일 형식의 엔디언을 정하면 바이트를 명시적으로 분리

```
void WriteUint32LE(std::ostream& output, std::uint32_t value)
{
    std::array<unsigned char, 4> bytes{
        static_cast<unsigned char>(value & 0xFFu),
        static_cast<unsigned char>((value >> 8) & 0xFFu),
        static_cast<unsigned char>((value >> 16) & 0xFFu),
        static_cast<unsigned char>((value >> 24) & 0xFFu)
    };
    output.write(
        reinterpret_cast<const char*>(bytes.data()),
        static_cast<std::streamsize>(bytes.size()));
}
```

- 문자열은 길이와 바이트를 함께 기록
- 읽을 때 길이 상한 검사 필요



15.5.5 객체를 그대로 저장할 때의 문제

```
struct PlayerRecord
{
    int id;
    std::string name;
    int hp;
};
PlayerRecord player{1, "Knight", 100};
output.write(reinterpret_cast<const char*>(&player), sizeof(player));
```

- `std::string` 객체에는 문자 데이터가 아니라 관리 정보와 포인터 포함 가능
- 다시 읽어 객체로 사용하면 불변식과 소유권이 깨짐

// 단순 구조체도 문제 가능

```
struct SimpleRecord
{
    std::uint32_t id;
    std::uint16_t level;
    std::uint32_t score;
};
```

- 멤버 사이 패딩
- 정수 바이트 순서
- 컴파일러 옵션과 정렬 규칙 차이
- 파일에는 객체 메모리가 아니라 복원에 필요한 논리적 필드 기록



- 지속적으로 사용하는 파일은 종류와 버전을 식별해야 함
- 필드 순서만 정하면 확장과 오류 검증이 어려움

```
output.write("PLYR", 4);
WriteUint32LE(output, 1);

std::array<char, 4> magic{};
input.read(magic.data(), magic.size());
if (!input || magic != std::array<char, 4>{'P', 'L', 'Y', 'R'}) {
    throw std::runtime_error{"Invalid file type"};
}

auto version = ReadUint32LE(input);
switch (*version) {
case 1:
    LoadVersion1(input);
    break;
case 2:
    LoadVersion2(input);
    break;
default:
    throw std::runtime_error{"Unsupported file version"};
}
```

- 필드 타입, 크기, 바이트 순서, 인코딩, 개수, 버전, 오류 검증 방법 정의



- `<filesystem>`은 경로, 파일, 디렉터리와 파일 속성 처리
- 문자열 직접 결합보다 `std::filesystem::path` 사용
- 운영체제별 경로 구분과 구성 요소를 타입으로 처리

```
#include <filesystem>

namespace fs = std::filesystem;

fs::path directory = "save";
fs::path fileName = "player.dat";
fs::path fullPath = directory / fileName;

// 연산자는 운영체제에 맞는 경로 구분자로 결합

fs::path path = "assets/characters/knight.png";
std::cout << path.filename() << '\n';
std::cout << path.stem() << '\n';
std::cout << path.extension() << '\n';
std::cout << path.parent_path() << '\n';
```



15.6.1 path 조작

```
fs::path path = "data/player.txt";  
path.replace_extension(".bin");  
path.replace_filename("monster.bin");  
  
fs::path relative = "data/player.txt";  
fs::path absolute = fs::absolute(relative);  
  
fs::path path = "assets/characters/../items/potion.png";  
fs::path normalized = path.lexically_normal();
```

함수	의미
filename()	파일 이름
stem()	확장자를 제외한 이름
extension()	확장자
parent_path()	부모 경로
absolute()	절대 경로
lexically_normal()	어휘 규칙으로 경로 정리

- 실제 파일 위치와 심볼릭 링크까지 반영하려면 **canonical()** 계열 사용
- 파일 시스템 접근과 오류 처리 필요



15.6.2 파일과 디렉터리 확인

```
fs::path path = "data/player.txt";  
if (fs::exists(path))  
{  
    if (fs::is_regular_file(path))  
    {  
        std::cout << "regular file\n";  
    }  
    else if (fs::is_directory(path))  
    {  
        std::cout << "directory\n";  
    }  
}
```

```
std::uintmax_t size = fs::file_size(path);  
auto writeTime = fs::last_write_time(path);
```

```
// error_code 오버로드  
std::error_code error;  
bool exists = fs::exists(path, error);  
if (error)  
{  
    std::cerr << error.message() << '\n';  
}  
else if (!exists)  
{  
    std::cout << "File not found\n";  
}
```

- 권한 문제와 파일 없음은 구분 필요 가능



```
fs::create_directory("save");
fs::create_directories("save/profile/slot1");
if (fs::create_directories("save/profile/slot1")) {
    std::cout << "Directory created\n";
}

// 직접 자식 순회
for (const fs::directory_entry& entry : fs::directory_iterator{"assets"}) {
    std::cout << entry.path() << '\n';
}

// 하위 디렉터리까지 순회
for (const fs::directory_entry& entry : fs::recursive_directory_iterator{"assets"})
{
    if (entry.is_regular_file()) {
        std::cout << entry.path() << '\n';
    }
}
```

- 순회 순서는 정렬 보장 없음
- 이름 순서가 필요하면 모은 뒤 정렬



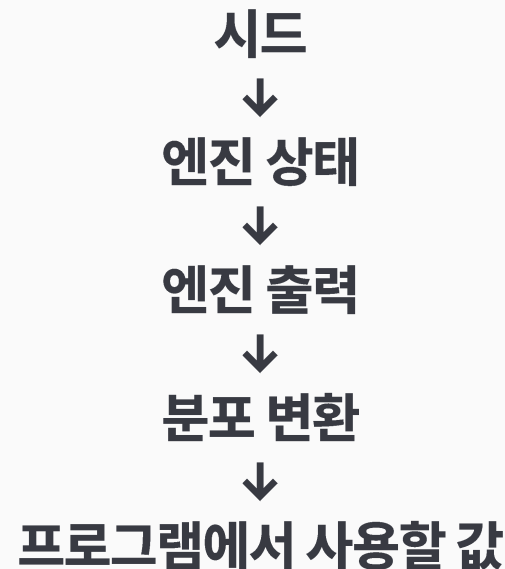
- 컴퓨터 난수는 대부분 결정적 계산으로 생성되는 의사 난수
- 같은 엔진 상태에서 시작하면 같은 수열 생성
- 난수 라이브러리는 엔진과 분포를 분리

```
#include <random>

std::mt19937 engine{1234};
std::uint32_t first = engine();
std::uint32_t second = engine();

std::mt19937 firstEngine{1234};
std::mt19937 secondEngine{1234};

assert(firstEngine() == secondEngine());
assert(firstEngine() == secondEngine());
```





```
#include <iostream>
#include <random>

int main() {
    std::mt19937 engine{1234};
    std::uniform_int_distribution<int> dice{1, 6};

    for (int i = 0; i < 10; ++i) {
        std::cout << dice(engine) << ' ';
    }
}
```

- `uniform_int_distribution<int>{1, 6}` 은 양 끝 포함
- 엔진은 넓은 정수 범위 생성
- 분포는 원하는 의미의 값으로 변환

```
std::uniform_real_distribution<double> ratio{0.0, 1.0};
double value = ratio(engine);
```

// 피함

```
int value = static_cast<int>(engine() % 6) + 1;
```

- `%` 방식은 균등 분포를 깨뜨릴 수 있음



```
std::mt19937 engine{2026};
```

// 시드는 엔진의 초기 상태 결정
// 고정 시드는 실행마다 같은 수열 필요할 때 사용

```
std::random_device randomDevice;  
std::mt19937 engine{randomDevice()};
```

- 실행마다 다른 시작 상태가 필요할 때 시드 재료 획득
- random_device를 매 난수마다 새로 호출하지 않음

```
std::seed_seq seed{  
    randomDevice(),  
    randomDevice(),  
    randomDevice(),  
    randomDevice()  
};
```

```
std::mt19937 engine{seed};  
int RollDice(std::mt19937& engine) {  
    std::uniform_int_distribution<int> dice{1, 6};  
    return dice(engine);  
}
```

- 엔진은 상태를 가진 객체
- 하나의 수열을 계속 사용하려면 참조 전달



- 버그 재현에는 시드와 호출 순서 기록이 효율적
- 같은 엔진 종류, 시드, 분포 매개변수, 호출 순서 필요
- 코드 변경으로 난수 호출 횟수가 바뀌면 결과가 달라질 수 있음

```
std::uint32_t seed = 20260802;
std::mt19937 engine{seed};

std::clog << "Random seed: " << seed << '\n';

// 엔진 상태 저장과 복원
std::ofstream output{"random_state.txt"};
output << engine;

std::ifstream input{"random_state.txt"};
input >> engine;

// 테스트를 위한 엔진 주입
class DamageCalculator {
public:
    explicit DamageCalculator(std::mt19937& engine) : engine(engine) {}
private:
    std::mt19937& engine;
};
```



```
std::uniform_int_distribution<int> integerRange{1, 100};
std::uniform_real_distribution<double> realRange{0.0, 1.0};
std::bernoulli_distribution criticalHit{0.2};
std::normal_distribution<double> height{170.0, 8.0};

if (criticalHit(engine)) {
    damage *= 2;
}

std::vector<int> cards{1, 2, 3, 4, 5};
std::shuffle(cards.begin(), cards.end(), engine);

// 무작위 원소 선택
if (!items.empty()) {
    std::uniform_int_distribution<std::size_t> index {0, items.size() - 1};
    Item& selected = items[index(engine)];
}
```

- 빈 컨테이너에서는 먼저 원소 존재 여부 확인
- 분포는 데이터의 실제 의미에 맞게 선택





15.7.6 rand와 random 라이브러리

```
#include <cstdlib>
```

```
#include <ctime>
```

```
std::srand(static_cast<unsigned int>(std::time(nullptr)));
```

```
int value = std::rand();
```

```
int dice = std::rand() % 6 + 1;
```

항목	std::rand()	<random>
상태	전역 상태	엔진 객체별 상태
범위 변환	사용자가 직접 처리	분포 객체가 처리
재현성	전역 호출 순서에 의존	시드와 엔진 명시 관리
알고리즘 선택	구현에 의존	엔진 타입 선택 가능
확률 분포	별도 구현 필요	여러 표준 분포 제공

- 새로운 C++ 코드에서는 <random> 우선
- 엔진을 적절한 소유 객체에 두고 필요한 함수에 참조로 전달



- `std::chrono`는 시간을 단위와 기준을 포함한 타입으로 표현
- 시간의 길이: `duration`
- 특정 시점: `time_point`
- 시점을 생성하는 기준: 시계 타입

```
#include <chrono>
```

```
std::chrono::seconds seconds{3};
```

```
std::chrono::milliseconds milliseconds{1500};
```

```
std::chrono::microseconds microseconds{250};
```

```
std::cout << seconds.count() << '\n';
```

```
std::cout << milliseconds.count() << '\n';
```

```
using namespace std::chrono_literals;
```

```
auto total = 2s + 500ms;
```

```
auto frame = 16ms;
```

```
auto timeout = 5s;
```

- 서로 다른 단위끼리 연산하면 공통 단위로 변환



15.8.2 time_point

```
using Clock = std::chrono::steady_clock;
```

```
Clock::time_point start = Clock::now();
```

```
Clock::time_point end = Clock::now();
```

```
auto elapsed = end - start;
```

- 두 시점의 차이는 duration

```
auto deadline = Clock::now() + std::chrono::seconds{5};
```

```
if (Clock::now() >= deadline)
```

```
{  
    std::cout << "Timeout\n";  
}
```

```
std::chrono::steady_clock::time_point steadyNow = std::chrono::steady_clock::now();
```

```
std::chrono::system_clock::time_point systemNow = std::chrono::system_clock::now();
```

- 서로 다른 시계의 time_point는 직접 비교하지 않음
- 같은 시계에서 얻은 시점끼리 차이를 계산





15.8.3 시계의 종류와 선택

시계	특성	적합한 용도
<code>std::chrono::system_clock</code>	시스템 달력 시각과 연결	파일 시각, 로그 시각, 외부 시간 표현
<code>std::chrono::steady_clock</code>	시간이 역행하지 않음	경과 시간, 제한 시간, 성능 측정
<code>std::chrono::high_resolution_clock</code>	짧은 틱 주기 목표로 구현 별칭	구현 특성을 확인한 특수 측정

```
using Clock = std::chrono::steady_clock;
```

```
auto start = Clock::now();
```

```
RunTask();
```

```
auto end = Clock::now();
```

```
static_assert(std::chrono::steady_clock::is_steady);
```

- `system_clock` 은 시간 동기화와 사용자 설정으로 조정 가능
- 실행 구간 길이 측정에는 `steady_clock` 사용
- `high_resolution_clock` 은 구현에 따라 다른 시계의 별칭일 수 있음



15.8.4 시간 단위 변환

```
using namespace std::chrono;
```

```
milliseconds value{2500};
```

```
seconds wholeSeconds = duration_cast<seconds>(value);
```

- wholeSeconds에는 2 저장
- 정수 단위 변환에서 남는 500ms는 절삭

```
auto secondsPart = duration_cast<seconds>(value);
```

```
auto millisecondsPart = value - secondsPart;
```

// 반올림 계열

```
auto down = std::chrono::floor<std::chrono::seconds>(value);
```

```
auto up = std::chrono::ceil<std::chrono::seconds>(value);
```

```
auto nearest = std::chrono::round<std::chrono::seconds>(value);
```

```
std::chrono::duration<double> secondsValue = value;
```

- 실수 단위 변환은 소수 부분 보존 가능
- 단위 변환은 표현 정밀도를 바꾸는 작업



```
#include <chrono>
#include <iostream>

int main() {
    using Clock = std::chrono::steady_clock;
    const auto start = Clock::now();

    RunTask();

    const auto end = Clock::now();
    const auto elapsed = end - start;

    const auto milliseconds = std::chrono::duration<double, std::milli>{elapsed};

    std::cout << milliseconds.count() << " ms\n";
}
```

- 시작과 종료 시점을 같은 **steady_clock** 에서 얻음
- 한 번의 측정은 캐시, 스케줄링, 다른 프로세스 영향 가능
- 짧은 함수는 여러 번 반복하고 평균 계산
- 결과가 사용되지 않는 계산은 컴파일러가 제거할 수 있음





- position += speed;
 - ◆ 프레임 횟수 기준이면 실행 속도에 따라 결과가 달라짐
- position += speed * deltaSeconds;
 - ◆ 속도가 초당 이동량 이면 경과 시간을 곱함

```
using Seconds = std::chrono::duration<double>;
```

```
void Update(Seconds deltaTime)  
{  
    position += speed * deltaTime.count();  
}
```

// 쿨다운

```
class Cooldown
```

```
{
```

```
public:
```

```
    explicit Cooldown(std::chrono::milliseconds interval) : interval(interval),  
        readyTime(std::chrono::steady_clock::time_point::min()) { }
```

```
private:
```

```
    std::chrono::milliseconds interval;
```

```
    std::chrono::steady_clock::time_point readyTime;
```

```
};
```

- 시간 길이와 완료 시점을 타입으로 구분



- 일정 간격 갱신에는 누적 시간 사용
- 프레임이 오래 지연되면 반복 횟수가 급격히 늘 수 있음
- 최대 프레임 시간 또는 한 프레임의 최대 업데이트 횟수 정책 필요

```
using Seconds = std::chrono::duration<double>;
```

```
Seconds accumulator{0.0};
```

```
constexpr Seconds fixedStep{1.0 / 60.0};
```

```
void Tick(Seconds frameTime)
{
    accumulator += frameTime;
    while (accumulator >= fixedStep) {
        FixedUpdate(fixedStep);
        accumulator -= fixedStep;
    }
}
```

```
#include <thread>
```

```
std::this_thread::sleep_for(std::chrono::milliseconds{100});
```

- 수면 함수는 정확한 실시간 스케줄을 보장하지 않음
- 운영체제 스케줄러 지연으로 더 늦게 깨어날 수 있음



- 스트림은 콘솔, 문자열, 파일을 공통된 읽기·쓰기 인터페이스로 다룸
- 입력 성공 여부는 실제 추출 연산으로 확인
- 입력 실패 복구에는 `clear()`와 `ignore()`가 함께 필요
- `operator>>`와 `std::getline()`을 함께 쓸 때는 남은 줄바꿈 처리
- 스트림 조정자 중 일부 상태는 이후 출력에도 유지
- `std::setw()`는 한 필드에만 적용
- `std::format()`은 호출 하나에 형식 규칙을 모아 문자열 생성
- `std::format_to()`는 기존 출력 대상에 기록
- 사용자 정의 타입은 스트림 연산자 또는 `std::formatter`로 출력 문법에 연결
- 문자열 스트림은 문자열 파싱과 조립에 유용





- 텍스트 파일은 사람이 읽기 쉽지만 구분자, 인코딩, 버전 규칙 필요
- 바이너리 모드는 텍스트 변환을 막을 뿐 객체를 자동 직렬화하지 않음
- 지속 가능한 바이너리 형식은 필드 크기, 바이트 순서, 문자열 길이, 버전을 명시
- 포인터와 동적 메모리를 포함한 객체를 메모리 그대로 저장하면 안 됨
- `std::filesystem::path`는 운영체제 경로를 타입으로 표현
- 파일 시스템 함수는 예외 방식과 `std::error_code` 방식 제공
- 파일은 검사 직후에도 외부 요인으로 변경될 수 있음
- 실제 작업 실패 처리가 필요





- 난수 라이브러리는 엔진과 분포를 분리
- 엔진은 상태를 가진 의사 난수 수열 생성
- 분포는 원하는 범위와 확률 모델로 변환
- 고정 시드는 테스트와 재현에 사용
- 실행마다 다른 수열이 필요하면 시드 재료를 외부에서 얻음
- 엔진 결과에 %를 적용하기보다 표준 분포 사용
- `std::chrono::duration`은 시간의 길이 표현
- `time_point`는 특정 시계의 시점 표현
- 달력 시각은 `system_clock`
- 경과 시간과 제한 시간은 `steady_clock`
- 시간 기반 처리는 프레임 횟수가 아니라 실제 경과 시간 기준

